UAB
Universitat Autònoma de Barcelona

Projecte 2

Intel·ligència Artificial

Roger Rodoreda Jubany - 1669698
Max Matarrodona Llopart - 1668784

Índex

Introducció 3

- Kmeans: 3
- KNN: 3

Mètodes d'anàlisi implementats 4

- Funcions d'anàlisi qualitatiu 4
- Funcions d'anàlisi quantitatiu 9

Millores sobre Kmeans i Knn 12

- Inicialització de centroides de Kmeans 12
- Diferents heurístiques per Find Best K 16
- Find_BestK 16

Conclusió global 18

Introducció

En aquest projecte ens plantegen el problema d'etiquetar una imatge segons la seva forma i colors.

Aquest problema, tot i que és molt fàcil pels humans, es complica més quan es tracta d'IA, per exemple en quantitat de càlculs. Tot i així es pot implementar mitjançant els algoritmes Kmeans pel color i KNN per la forma, sempre i quan tinguem les suficients mostres per reduïr l'error.

- Kmeans:
  Algorisme que busca clusters de punts sense etiquetar en un espai. Per fer això, donat un nombre de clusters k que hem calculat prèviament mitjançant la funció find_bestK(), va recalculant els centres dels clusters fins que els nous centroides no són diferents dels anteriors.

  En aquest punt, podem classificar els centroides, assignant un color segons la posició on estan. Al fer això, podem trobar els colors que corresponen a la imatge, ja que serà dels colors assignats als centroides.

- KNN:
  Algorisme que donat un conjunt d'imatges etiquetades i una nova imatge, calcula totes les distàncies entre la imatge a etiquetar i tot el conjunt de punts etiquetats i els ordena de menys a més distància.
  Per últim, la classe que assignarem serà aquella que apareixi més vegades als K primers punts (que hem ordenat prèviament).

Mètodes d'anàlisi implementats

A part de les funcions necessàries a implementar especificades al document, hem implementat alguna funció extra per tal de facilitar les proves, així que començarem explicant-les.

- Execute_kmeans_images(image_set, k_max, init = {}): Donat un conjunt d'imatges, una k màxima i opcions d'inicialització (per defecte està buit), retorna les imatges transformades a la classe Kmeans amb els paràmetres inicialitzats.

- Execute_classifications(image_set,image_train, class_train,k_max = 10, k_knn=4, init = {}): Donat un conjunt d'imatges, un conjunt d'imatges d'entrenament, un conjunt de classes associades a les imatges d'entrenament, una k màxima (Kmeans), una k màxima (per Knn) i opcions d'inicialització (per defecte està buit), retorna les imatges transformades a la classe Kmeans amb els paràmetres inicialitzats, els colors associats a cada imatge i les classes associades a cada imatge.

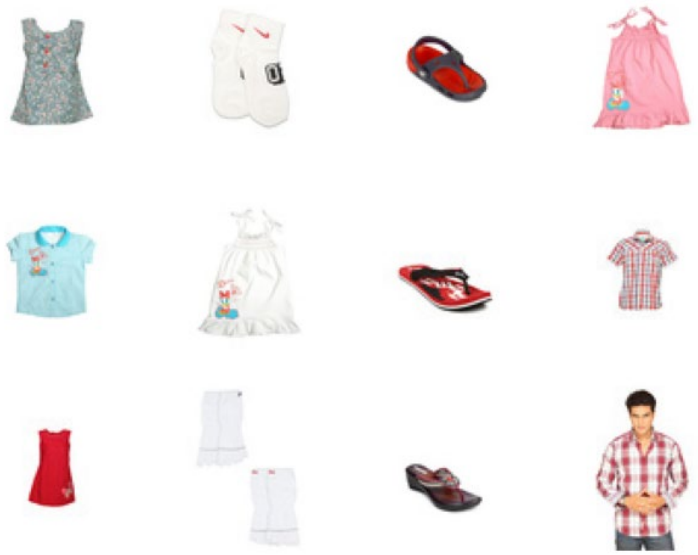
- Funcions d'anàlisi qualitatiu

Aquestes són les funcions que hem fet per calcular el Knn i el Kmeans utilitzant el nostre codi. Ara explicarem amb més detall les funcions que hem hagut d'implementar.

- Retrieval_by_color(images, colors, query, or_cond = True): En aquesta funció, donat un conjunt d'imatges, els colors associats, una cerca retorna una llista d'imatges que continguin el o els colors de la query i si volem que contingui un o tots els colors que li passem (or_cond = True/False. Per defecte a True). Per tal d'aconseguir això, per cada query iterem totes les imatges, mirant si els colors que corresponen a aquesta imatge contenen el o els colors de la query. Puntualització: per dos colors retorna les imatges que contenen com a mínim un dels dos colors.

Exemple d'execució, pel color "Red, Green" per defecte:

```
combo_images_retrival = Retrieval_by_color(imgs, color_labels, ["Red","Green"]
visualize_retrieval(combo_images_retrival, 12)
```



Exemple d'execució, pel color "Red, Green" i or_cond = False:

```
combo_images_retrival = Retrieval_by_color(imgs, color_labels, ["Red","Green"], or_cond = False)
visualize_retrieval(combo_images_retrival, 12)
```

- Retrieval_by_shape(images, shape, query):

  Donat un conjunt d'imatges amb la seva forma corresponent i una query, la funció retorna totes aquelles imatges que tinguin la forma o una de les formes de la query. Primer recorrem les querys, iterant totes les imatges per cada una i comprovant si són de la classe consultada.
  Retorna totes les fotos que tinguin la forma consultada.

  Exemple d'execució amb "Jeans" i "Socks" (Es retorna en l'ordre de consulta):

```
label_images_retrival = Retrieval_by_shape(test_imgs, test_class_labels, ["Jeans", "Socks"])
visualize_retrieval(label_images_retrival, 12)
```

```
label_images_retrival = Retrieval_by_shape(test_imgs, test_class_labels, ["Socks", "Jeans"])
visualize_retrieval(label_images_retrival, 12)
```

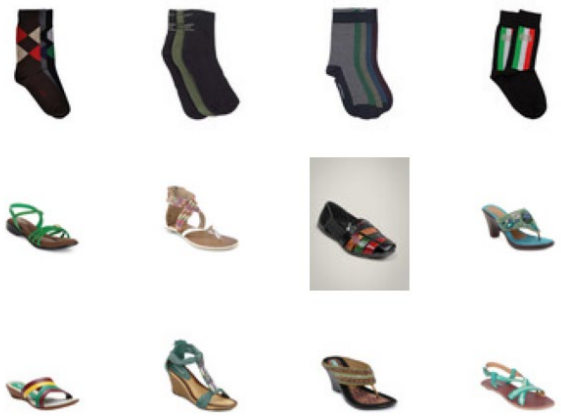
- Retrival_combined(images, color, shape, color_querys, shape_querys):

  Donat un conjunt d'imatges juntament amb els seus respectius colors i formes, a més d'una consulta conformada per com a mínim una parella del tipus Color + Forma, retorna totes les imatges que siguin del color i forma desitjats.

  Exemple d'execució:

```
combo_images_retrival = Retrieval_combined(test_imgs, test_color_labels, test_class_labels, ["Green", "Blue"], ["Socks", "Jeans"])
visualize_retrieval(combo_images_retrival, 12)
```

```
combo_images_retrival = Retrieval_combined(test_imgs, test_color_labels, test_class_labels, ["Green", "Green"], ["Socks", "Heels"])
visualize_retrieval(combo_images_retrival, 12)
```



Com es pot veure, l'execució és l'esperada. Podem veure com ens retorna les imatges de la query desitjada. Cal puntualitzar que les demostracions adjuntades al document les hem fet mitjançant el conjunt d'imatges "test_images", ja que ja tenen els colors i formes assignades. Gràcies això, hem pogut determinar correctament el funcionament de les funcions. Així doncs, proporcionem algunes cerques utilitzant la classificació del nostre codi:

```
my_image_to_km, my_image_color, my_image_shape = Execute_classifications(imgs,train_imgs,train_class_labels, k_max = 10, k_knn=4, init = {})
combo_images_retrival = Retrieval_by_color(imgs, my_image_color, ["Red","Green"], or_cond = False)
visualize_retrieval(combo_images_retrival, 12)
label_images_retrival = Retrieval_by_shape(imgs, my_image_shape, ["Socks", "Jeans"])
visualize_retrieval(label_images_retrival, 12)
combo_images_retrival = Retrieval_combined(imgs, my_image_color, my_image_shape, ["Green", "Grey"], ["Socks", "Heels"])
visualize_retrieval(combo_images_retrival, 12)
```

- Funcions d'anàlisi quantitatiu
- def Kmean_statistics(km, Kmax):

  Donats un objecte de la classe Kmeans (una imatge o conjunt d'imatges) i una k màxima, calcula la WCD, el nombre d'iteracions per convergència, el temps d'execució.

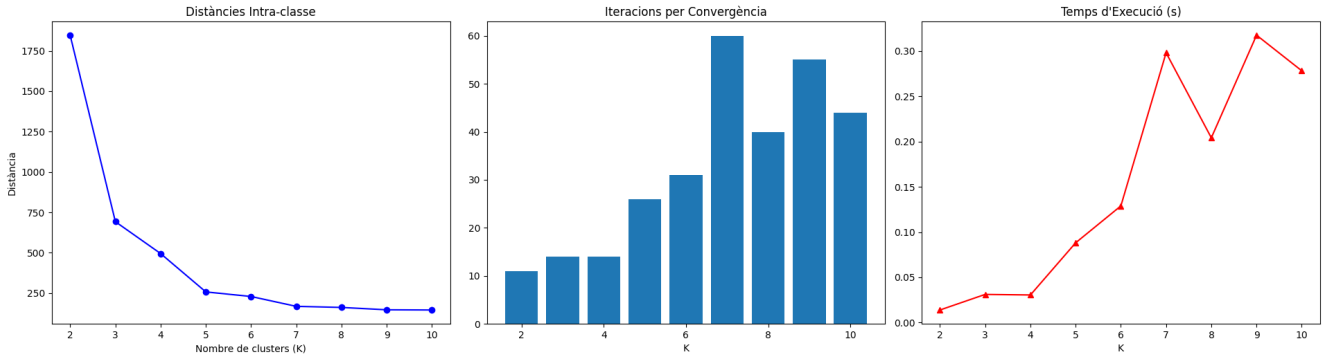
  Exemple d'execució amb Kmax = 10:

```
from Kmeans import KMeans
km = KMeans(test_imgs[4], K=2)

km.options['fitting'] = 'WCD'
km.find_bestK(10)
print(f"Millor K {km.options['fitting']}: {km.K}")

Kmean_statistics(km, Kmax=10)
```

```
Millor K WCD: 5
```



  Observem que find_best_k troba que la millor k seria k = 5, que es troba al colze de la corba de les distàncies intra-classe (WCD).

- def Get_shape_accuracy(prediccions, ground_truth):

  Donades les etiquetes predites en aplicar KNN i el "Ground-Truth" d'aquestes, retorna el percentatge d'etiquetes que hem predit correctament.

  Exemple d'execució:

  Aquest codi executa la funció en sí i també la visualització de les dades de forma que quedin més clarament presentades. Cal remarcar que hem comprovat que per aquest conjunt d'imatges la K òptima és 4.

```python
#execució del KNN per obtenir prediccions
trained_knn = KNN(utils.rgb2gray(test_imgs), test_class_labels)
predicted_shapes = trained_knn.predict(utils.rgb2gray(test_imgs), k=4)

#càlcul de l'encert
accuracy = Get_shape_accuracy(predicted_shapes, test_class_labels)
total_samples = len(test_class_labels)
error_count = np.sum(np.array(predicted_shapes) != np.array(test_class_labels))

print(f"Precisió en la classificació de formes: {accuracy:.2f}%")
print(f"Errors: {error_count}/{total_samples}")

#visualització d'exemples incorrectes amb més informació
incorrectes = np.where(np.array(predicted_shapes) != np.array(test_class_labels))[0]
if len(incorrectes) > 0:
    #preparem la informació ampliada
    error_info = [
        f"Predicció: {predicted_shapes[i]}\nReal: {test_class_labels[i]}"
        for i in incorrectes[:8]
    ]

    #creem el títol amb les estadístiques
    plot_title = (
        f"Classificacions incorrectes\n"
        f"Precisió: {accuracy:.2f}% | Errors: {error_count}/{total_samples}"
    )

    visualize_retrieval(
        test_imgs[incorrectes[:8]],
        topN=8,
        info=error_info,
        ok=[False]*8,
        title=plot_title,
        query=None
    )
```

- def Get_color_accuracy(prediccions, ground_truth):

  Donats els colors predits en aplicar el Kmeans i el “Ground-Truth” d’aquestes, retorna el percentatge de colors predits correctament.

  Per avaluar si els colors predits coincideixen amb els reals, hem decidit utilitzar l'índex de Jaccard, que mesura la similitud entre dos conjunts. Aquesta mètrica compara quants colors coincideixen (intersecció) respecte al total de colors únics presents (unió).

Exemple d'execució:

Hem posat el resultat de la funció i 5 exemples concrets de predicció de colors. Cal remarcar que hem comprovat que la millor k per aquest conjunt d'imatges és 5.

```python
predicted_colors = []

#processar totes les imatges de test
for img in test_imgs:
    km = KMeans(img, K=5)
    km.fit()
    predicted_colors.append(get_colors(km.centroids))

#calcular precisió total
print(f"Analitzant {len(test_imgs)} imatges\n")
color_accuracy = Get_color_accuracy(predicted_colors, test_color_labels)
print(f"\nSimilitud mitjana total en colors: {color_accuracy:.2f}%")

for i in range(5):
    #calcular Jaccard individual
    set_pred = set(predicted_colors[i])
    set_real = set(test_color_labels[i])
    interseccio = len(set_pred & set_real)
    unio = len(set_pred | set_real)
    similitud = (interseccio / unio) * 100 if unio > 0 else 100

    #visualització
    print(f"Imatge {i}:")
    print(f"  Predicció: {', '.join(predicted_colors[i])}")
    print(f"  Real:      {', '.join(test_color_labels[i])}")
    print(f"  Coincidències: {interseccio} de {unio} colors")
    print(f"  Similitud: {similitud:.2f}%")
    print("-"*50)
```

```
Analitzant 851 imatges


Similitud mitjana total en colors: 37.89%
Imatge 0:
  Predicció: White, Grey, Black, Grey, White
  Real:      Black, White
  Coincidències: 2 de 3 colors
  Similitud: 66.67%
--------------------------------------------------
Imatge 1:
  Predicció: White, Grey, Black, Black, Grey
  Real:      Blue, Brown
  Coincidències: 0 de 5 colors
  Similitud: 0.00%
--------------------------------------------------
Imatge 2:
  Predicció: White, Orange, Red, Grey, Black
  Real:      Blue
  Coincidències: 0 de 6 colors
  Similitud: 0.00%
--------------------------------------------------
Imatge 3:
  Predicció: White, Orange, Grey, Brown, Black
  Real:      Black
  Coincidències: 1 de 5 colors
  Similitud: 20.00%
--------------------------------------------------
Imatge 4:
  Predicció: White, Orange, Pink, Blue, Blue
  Real:      Blue
  Coincidències: 1 de 4 colors
  Similitud: 25.00%
--------------------------------------------------
```

Millores sobre Kmeans i Knn

- ## Inicialització de centroides de Kmeans

En aquesta millora, hem creat diferents maneres per inicialitzar els centroides per executar el fit() de Kmeans.

Hem creat tres noves maneres:

1. **Grey**: Donada una imatge, calcula el valor mínim i màxim de cada dimensió dels píxels (proper igual a (255,255,255) [max] i (0,0,0)[min]) i inicialitza k punts en la diagonal entre el màxim i el mínim. Per exemple, per k=3, podria inicialitzar els centroides a (255,255,255),(127,127,127),(0,0,0).

   Com que la majoria de punts de la imatge són propers a la diagonal de grisos, és més fàcil trobar els clústers si els punts ja estan inicialitzats en aquesta línia.

2. **Color_prob**: Donat un conjunt gran d'imatges (en aquest cas hem utilitzat "train_images"), calculem la quantitat de cada color, obtenint com a resultat quins són els colors més comuns.
   Utilitzant aquesta informació, inicialitzarem els centroides a les coordenades RGB dels colors més probables primer, ja que així ja tindrem el centroide al lloc més proper a un possible cluster.
   Per exemple, donat k=5, inicialitzarem els centroides a:
   [255.0, 255.0, 255.0], *#White: 2328*
   [128.0, 128.0, 128.0], *#Grey: 1242*
   [0.0, 0.0, 0.0], *#Black: 1148*
   [255.0, 165.0, 0.0], *#Orange: 1030*
   [0.0, 0.0, 255.0], *#Blue: 555*

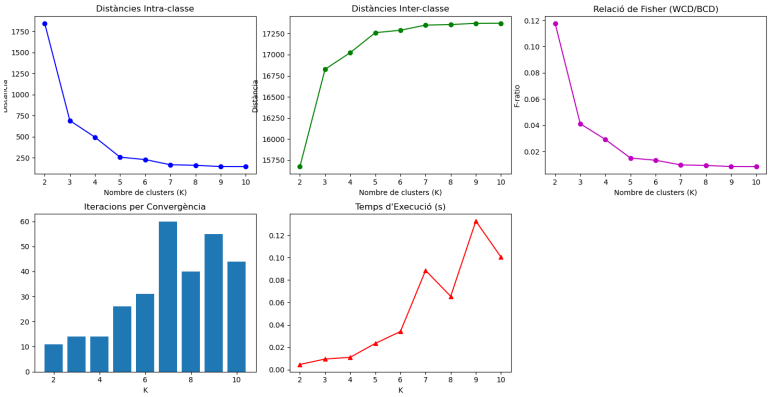
3. **Color_distances (o dispersed)**: Utilitzant la mateixa lògica que a "Color_prob", inicialitzem els centroides a les coordenades RGB dels possibles colors, ja que els clusters hauríen d'estar a prop d'aquestes coordenades. En aquest cas inicialitzarem primer els punts que siguin més llunyans entre sí, ja que així els punts es distribuïran millor per tot l'espai. Així doncs, per k=6, inicialitzarem els centroides a:
   [255.0, 255.0, 255.0], *#White: 2328*
   [0.0, 0.0, 0.0], *#Black: 1148*
   [128.0, 128.0, 128.0], *#Grey: 1242*
   [0.0, 0.0, 255.0], *#Blue: 555*
   [255.0, 0.0, 0.0], *#Red: 361*
   [0.0, 255.0, 0.0], *#Green: 171*

A partir d'aquí, si executem totes les inicialitzacions:

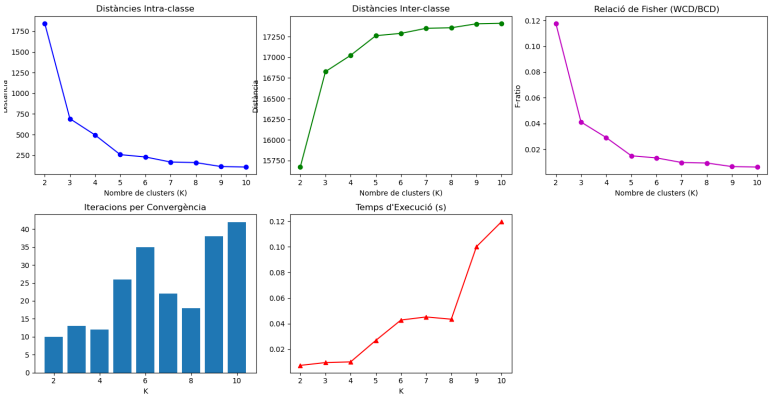
```python
for available_option in avaliable_options:
    km = KMeans(test_imgs[4], K=2, options=available_option)
    Kmean_statistics(km, Kmax=10)

    H, W, C = test_imgs[4].shape
    visualize_k_means(km, (H, W, C))
```

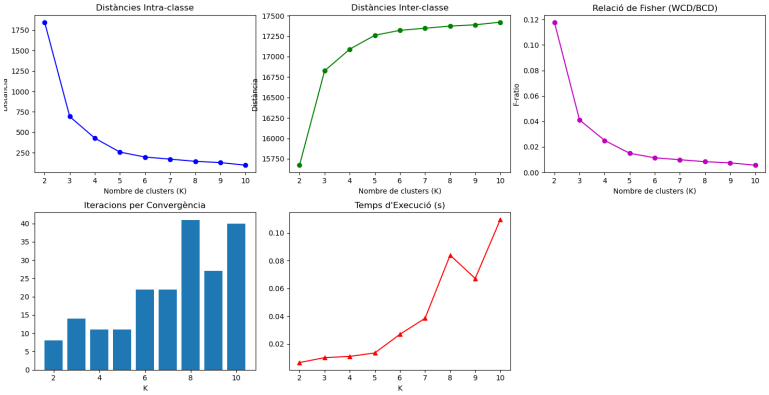
First:

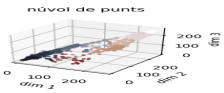


Random:

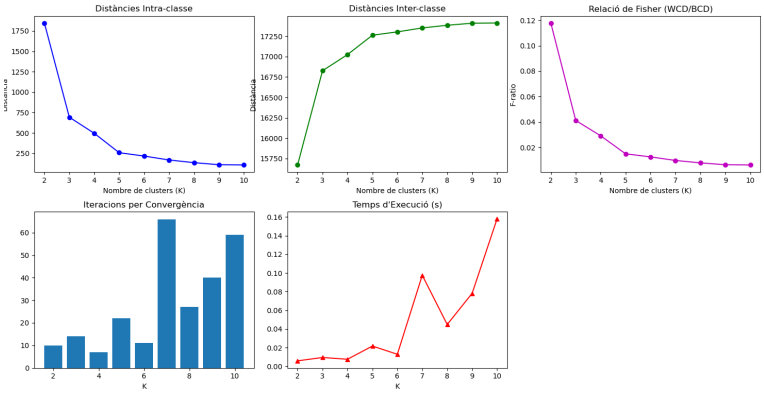


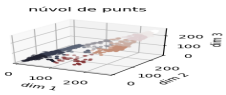
Grey:



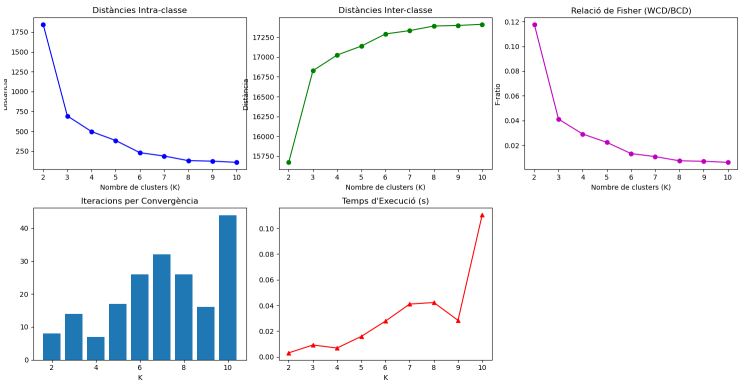


Color_prob:





Color_distances:



```
for available_option in avaliable_options:
    km_images, colors, shapes = Execute_classifications(imgs,train_imgs,train_class_labels,k_max = 10, k_knn=4, init = available_option)
    color_accuracy = Get_color_accuracy(colors, color_labels)
    print(f"Opció: {available_option}")
    print(f"Similitud mitjana total en colors: {color_accuracy:.2f}%\n")
```

```
Opció: {'km_init': 'first', 'verbose': False, 'tolerance': 0, 'max_iter': inf, 'fitting': 'WCD'}
Similitud mitjana total en colors: 42.58%

Opció: {'km_init': 'random', 'verbose': False, 'tolerance': 0, 'max_iter': inf, 'fitting': 'WCD'}
Similitud mitjana total en colors: 43.38%

Opció: {'km_init': 'grey', 'verbose': False, 'tolerance': 0, 'max_iter': inf, 'fitting': 'WCD'}
Similitud mitjana total en colors: 44.46%

Opció: {'km_init': 'color_prob', 'verbose': False, 'tolerance': 0, 'max_iter': inf, 'fitting': 'WCD'}
Similitud mitjana total en colors: 43.71%

Opció: {'km_init': 'color_dispersed', 'verbose': False, 'tolerance': 0, 'max_iter': inf, 'fitting': 'WCD'}
Similitud mitjana total en colors: 43.18%
```

Veient els resultats, veiem com amb la opció first, els detalls petits de colors no es veuen reflectits al resultat (el cinturó passa a ser gris), mentres que totes les altres maneres d'inicialitzar sí que troben el vermell com a color present a la imatge.
Per altra banda, el temps d'execució de "first" és més gran que al resta.
El temps de "random" pot variar segons on apareixin els punts.
"grey" va augmentant el temps de manera que com més gran és k, més tarda.
"color_prob" tarda molt poc temps per valors de k inferiors a 6 i bastant més temps per valors més grans, també baixa per k=8 (provablament perquè el color nº8 estava a la imatge).
"Color distances" va augmentant fins a 7 o 8, baixa el temps a k=9 (provablament perquè el color nº9 estava a la foto)

Pel que fa al percentatge d'encerts, el millor resultat és el de "Grey", seguit de "Clor_prob", "Color_dispersed", "random" i "first".
Així doncs, podem arribar a la conclusió que, en general, serà millor inicialitzar amb "Grey".

- ## Diferents heurístiques per Find_BestK

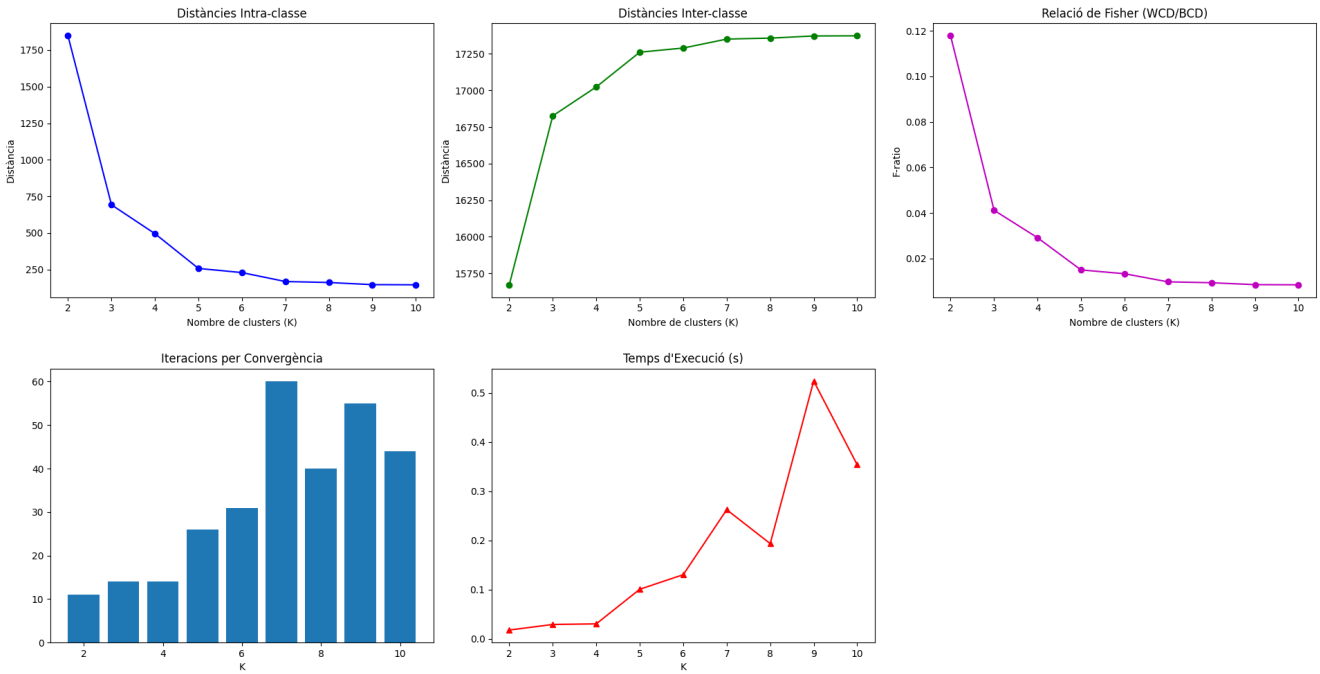
  A més de l'heurística "Within Class Distance", n'hem programat dues més: "Between Class Distance" (Calcula la distància entre les diferents classes) i "Fisher Information Metric" (Mesura la variabilitat dins de cada classe respecte la distància entre classes, WCD/BCD).
  Havent creat aquestes heurístiques, també hem decidit millorar la funció Kmean_statistics afegint dos gràfics amb les mètriques BCD i Fisher.

  Exemple d'execució de la nova versió de Kmean_statistics:

```
from Kmeans import KMeans
km = KMeans(test_imgs[4], K=2)

km.options['fitting'] = 'WCD'
km.find_bestK(10)
print(f"Millor K {km.options['fitting']}: {km.K}")

Kmean_statistics(km, Kmax=10)
```



- ## Find_BestK

  Utilitzant les heurístiques que hem creat, hem modificat la funció Find_BestK per afegir-les com a opció de "fitting": WCD, BCD i Fisher. Valor del llindar, respectivament: 15, 0.1, 15.

  Exemple d'execució de la funció utilitzant les tres heurístiques:

```python
from Kmeans import KMeans
km = KMeans(test_imgs[4], K=2)

km.options['fitting'] = 'WCD'
km.find_bestK(10)
print(f"Millor K {km.options['fitting']}: {km.K}")

km.options['fitting'] = 'BCD'
km.find_bestK(10)
print(f"Millor K {km.options['fitting']}: {km.K}")

km.options['fitting'] = 'FISHER'
km.find_bestK(10)
print(f"Millor K {km.options['fitting']}: {km.K}")
```

```
Millor K WCD: 5
Millor K BCD: 5
Millor K FISHER: 5
```

Si comparem els resultats obtinguts amb el gràfic de Kmean_statistics de l'anterior apartat, podem observar que K = 5 coincideix amb els colzes dels gràfics, verificant que és una K òptima per aquesta imatge.

Addicionalment, hem combinat les noves opcions de find_bestK amb les de inicialització de centroides per trobar un millor resultat de "Color Accuracy", comprovant que amb "'km_init': 'color_dispersed'" i "'fitting': 'BCD'" obtenim el millor resultat possible.

```python
for type in ["WCD","BCD","FISHER"]:
    for available_option in avaliable_options:
        available_option['fitting'] = type
        km_images, colors, shapes = Execute_classifications(imgs,train_imgs,train_class_labels,k_max = 10, k_knn=4, init = available_option)
        color_accuracy = Get_color_accuracy(colors, color_labels)
        print(f"Opció: {available_option}")
        print(f"Similitud mitjana total en colors: {color_accuracy:.2f}%\n")
```

```
Opció: {'km_init': 'first', 'fitting': 'WCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 42.63%

Opció: {'km_init': 'random', 'fitting': 'WCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 42.24%

Opció: {'km_init': 'grey', 'fitting': 'WCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 45.08%

Opció: {'km_init': 'color_prob', 'fitting': 'WCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 44.15%

Opció: {'km_init': 'color_dispersed', 'fitting': 'WCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 43.83%

Opció: {'km_init': 'first', 'fitting': 'BCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 43.98%

Opció: {'km_init': 'random', 'fitting': 'BCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 44.42%

Opció: {'km_init': 'grey', 'fitting': 'BCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 45.14%

Opció: {'km_init': 'color_prob', 'fitting': 'BCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 45.11%

Opció: {'km_init': 'color_dispersed', 'fitting': 'BCD', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 45.29%

Opció: {'km_init': 'first', 'fitting': 'FISHER', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 42.58%

Opció: {'km_init': 'random', 'fitting': 'FISHER', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 43.46%

Opció: {'km_init': 'grey', 'fitting': 'FISHER', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 44.99%

Opció: {'km_init': 'color_prob', 'fitting': 'FISHER', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 44.58%

Opció: {'km_init': 'color_dispersed', 'fitting': 'FISHER', 'verbose': False, 'tolerance': 0, 'max_iter': inf}
Similitud mitjana total en colors: 43.81%
```

Conclusió global

Durant aquest projecte hem desenvolupat un etiquetador automàtic d'imatges que combina Kmeans per classificar colors i KNN per identificar formes, arribant a obtenir resultats prometedors. Mitjançant un anàlisi qualitatiu i quantitatiu, acompanyat d'unes millores finals, hem obtingut:

- Precisió en colors: Hem avaluat la similitud entre els colors predits i els reals ("Ground-Truth"), obtenint un resultat prou efectiu en arribar a coincidències parcials utilitzant només els colors, amb una exactitud de fins a un 45.29% una vegada aplicades les millores en inicialització i heurística.
- Identificació de formes: Utilitzant l'algorisme KNN, hem demostrat obtenir un resultat excel·lent en identificar les diferents categories de la roba, arribant a un percentatge d'encert del 94.36%.

Finalment, algunes possibles aplicacions del nostre projecte podrien ser:

- E-commerce: Filtratge automàtic de productes en cerques dins de botigues virtuals.
- Accessibilitat: Descripció automàtica d'imatges per a persones amb una discapacitat visual.
- Arxivament d'imatges: Organització per seccions de biblioteques visuals basada en colors i tipus de peces.